

Self-Driven Agent 1 Working

Project Overview

Self-Driven Agent 1 Working is an experimental project that explores how Large Language Models (LLMs) can assist with exploratory data analysis inside a Python notebook environment.

The project investigates an LLM-assisted data-analysis workflow in which a user can provide a pandas DataFrame and ask analytical questions in natural language. Instead of manually writing every analysis step, the notebook provides helper logic that connects the dataset with an LLM and uses the model to assist in interpreting and answering analytical questions.

The primary goal of the project is to explore how AI agents can act as an interactive layer between a data scientist and a structured dataset.

The project focuses on the following question:

Can an LLM be used as an interactive analytical assistant to help users explore, understand, and reason about tabular data through natural-language questions?

Motivation

Traditional data analysis generally requires users to manually perform several steps:

1. Load the dataset.
2. Inspect the schema.
3. Identify relevant variables.
4. Write filtering or aggregation code.
5. Generate visualizations.
6. Interpret the results.
7. Repeat the process for new questions.

An LLM-assisted workflow can potentially reduce the amount of repetitive analytical work by allowing users to express questions in natural language.

For example, instead of manually constructing a sequence of pandas operations, a user could ask questions such as:

"What are the most important patterns in this dataset?"

or:

"Which variables appear to have the strongest relationship with the target?"

The LLM can then assist with the reasoning process and return an analytical response.

This project is an initial experiment toward building such a workflow.

Research / Engineering Objective

The main objective is to build and evaluate a lightweight notebook-based analytical agent capable of interacting with tabular data.

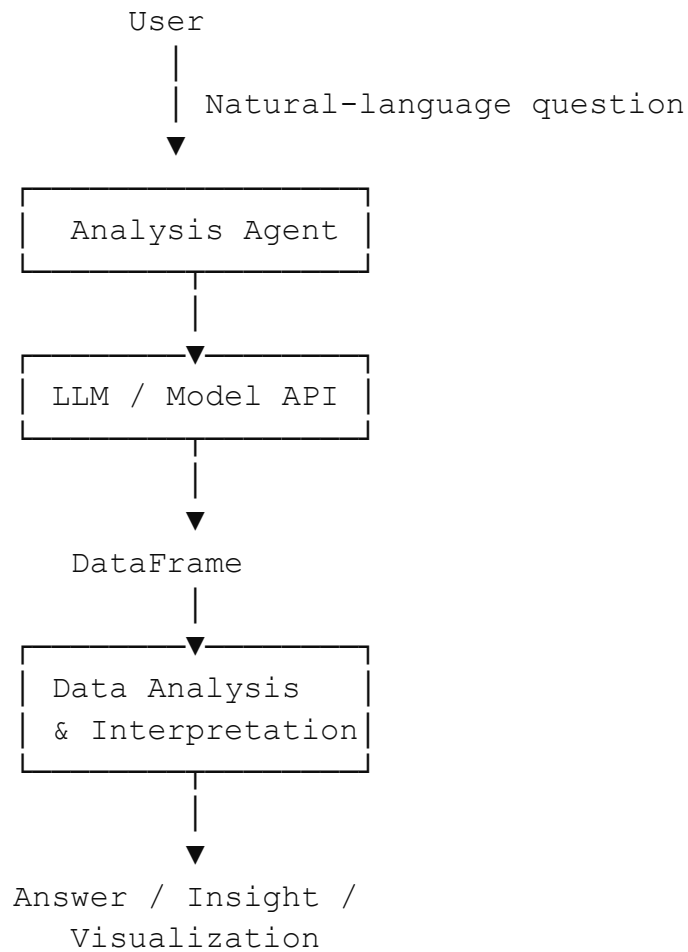
The project explores how an LLM can be integrated with common data-science tools such as:

- Pandas
- Plotting libraries
- Dataset utilities
- LLM APIs
- Python notebook environments

The current implementation focuses on experimentation rather than production deployment.

Core Concept

The project follows a simple architecture:



The notebook acts as the experimental environment where the different components are connected.

Workflow

The current notebook implements the following workflow.

1. Environment Setup

The notebook installs and imports the required Python dependencies.

The experimental environment includes libraries such as:

- pandas
- plotnine
- litellm
- datasets

These libraries provide the basic infrastructure for data manipulation, visualization, dataset access, and LLM integration.

2. Data Loading

The workflow uses a pandas DataFrame as the primary data representation.

The DataFrame acts as the structured source of information that the analytical agent reasons about.

This allows the system to work with standard tabular data rather than requiring a specialized database or application backend.

3. LLM Configuration

An LLM client is configured inside the notebook.

The model connection is responsible for processing analytical questions and generating responses based on the available dataset context.

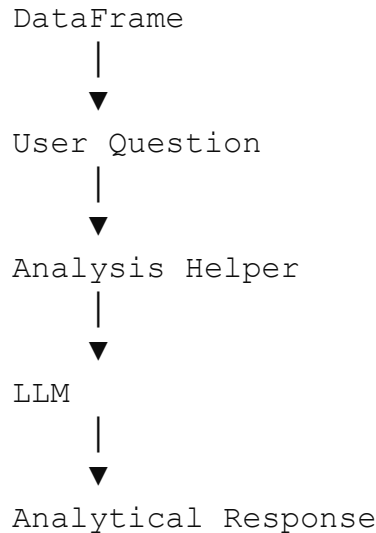
External API configuration may be required depending on the selected model provider.

Credentials and API keys should be supplied through environment variables or a secure configuration mechanism rather than being hard-coded into the notebook.

4. Analytical Questioning

The notebook defines helper functions that allow analytical questions to be passed to the model.

The basic interaction can be represented as:



This creates a natural-language interface for exploratory analysis.

Data Science Capabilities

The experiment is designed around several common exploratory data-analysis tasks.

Dataset Understanding

The LLM can assist in interpreting:

- Dataset structure
- Columns
- Data types
- Available variables
- General patterns

Exploratory Analysis

The workflow can be used to investigate:

- Distributions
- Relationships between variables
- Potential trends
- Interesting subsets
- Summary statistics

Analytical Question Answering

Users can ask targeted questions about the DataFrame and receive natural-language analytical responses.

Visualization Support

The notebook includes plotting functionality, allowing analysis results to be visualized alongside model-generated interpretations.

The combination of visual analysis and LLM reasoning is particularly useful for exploratory workflows where the analyst is iteratively investigating a dataset.

Technology Stack

The project uses the following technologies:

Technology	Purpose
Python	Core programming language
Jupyter Notebook	Interactive development and experimentation
Pandas	Data manipulation and tabular analysis
Plotnine	Data visualization
LiteLLM	LLM API abstraction and model interaction
Hugging Face Datasets / datasets	Dataset utilities
Logging	Experiment and execution monitoring

Repository Structure

The current repository is intentionally lightweight.

Self-Driven-Agent-1-Working/

```
|  
└─ test.ipynb
```

test.ipynb

The main experimental notebook.

It contains:

- Dependency setup
- Library imports
- Logging configuration
- LLM client initialization
- DataFrame analysis helpers
- Analytical question handling
- Visualization experiments
- Interactive analysis outputs

Example Workflow

A typical experiment follows this sequence:

1. Start Jupyter Notebook
- ↓
2. Install / import dependencies
- ↓
3. Load a dataset
- ↓
4. Convert data into a pandas DataFrame
- ↓
5. Configure the LLM
- ↓
6. Ask an analytical question
- ↓
7. Pass relevant dataset context to the LLM
- ↓
8. Generate analytical response
- ↓
9. Validate the result using pandas / visualization
- ↓
10. Iterate with another question

The important aspect of the workflow is the human-in-the-loop design.

The LLM is not intended to replace the data scientist. Instead, it acts as an analytical assistant while the user remains responsible for validating conclusions.

Why This Approach Is Useful

LLM-assisted data analysis can be valuable because data exploration is often an iterative process.

A data scientist may start with a broad question and progressively refine it:

```
"What does this dataset contain?"  
  ↓  
"Which variables are most important?"  
  ↓  
"Are there unusual observations?"  
  ↓  
"How does variable X relate to variable Y?"  
  ↓  
"Can we visualize this relationship?"  
  ↓  
"What could explain this pattern?"
```

An LLM can provide a conversational interface for this iterative process.

The notebook therefore explores the idea of treating an LLM as a natural-language analytical interface over structured data.

Important Design Principle: Human Validation

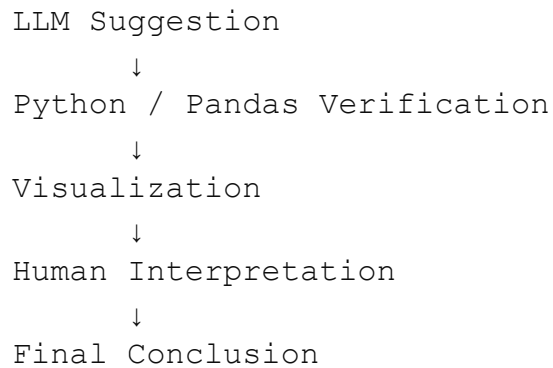
LLM-generated analysis should not automatically be treated as ground truth.

The model may:

- Misinterpret a column
- Make incorrect statistical assumptions
- Generate unsupported conclusions
- Misread correlations as causation
- Produce plausible but incorrect explanations

For this reason, the workflow should be treated as AI-assisted analysis, rather than fully autonomous statistical analysis.

A robust workflow is:



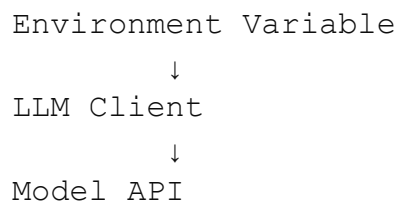
This combination provides a safer and more reliable analytical workflow.

Security and API Configuration

The project may require an external LLM API to execute the model-assisted portions of the notebook.

API credentials should never be committed to GitHub.

Recommended configuration:



For example, API keys should be stored using environment variables or a secure secret-management solution.

A .env file containing secrets should also be excluded through .gitignore.

Current Project Status

The repository is currently an experimental proof of concept.

The main implementation exists inside a single Jupyter notebook, making it useful for experimentation and rapid iteration.

The current version demonstrates:

- LLM integration
- DataFrame-based analysis
- Natural-language analytical questioning
- Basic visualization support
- Notebook-based experimentation

However, it is not yet structured as a production-ready analytical agent.

Current Limitations

Several areas require improvement before this project can be considered production-ready.

Notebook-Centric Architecture

Most functionality currently exists inside `test.ipynb`.

This makes reuse, testing, and maintenance more difficult.

Reproducibility

The project currently does not provide a dedicated environment specification such as:
`requirements.txt`

or:

`environment.yml`

External API Dependency

LLM-assisted functionality may require an external model provider and valid API credentials.

Limited Testing

There is currently no dedicated automated test suite for validating the analytical helper functions.

No Persistent Results

Generated analysis results are currently produced interactively rather than being stored in a structured output format.

Future Improvements

The project can be developed into a more complete AI-powered data-analysis system.

1. Modularize the Code

Move notebook logic into reusable Python modules.

A possible architecture would be:

```
src/  
├── data_agent/  
│   ├── __init__.py  
│   ├── data_loader.py  
│   ├── analyzer.py  
│   ├── llm_client.py  
│   ├── prompts.py  
│   ├── visualization.py  
│   └── validation.py
```

This would make the project easier to maintain and test.

2. Add a Requirements File

Create a reproducible environment using:

```
requirements.txt
```

or a modern dependency-management system such as Poetry or uv.

3. Add Automated Testing

Introduce tests for:

- Data loading
- DataFrame inspection
- LLM response handling
- Prompt construction
- Analytical helper functions
- Error handling

4. Improve Data Validation

Before sending data-related questions to the LLM, the system should validate:

- Column names
- Data types
- Missing values
- Dataset size
- Numeric versus categorical variables

This can reduce incorrect model interpretations.

5. Add Tool-Based Analysis

A more advanced agent could allow the LLM to request actual analytical operations.

For example:

```
User Question
      ↓
    LLM
      ↓
Select analytical tool
      ↓
Pandas / Python
      ↓
Execute calculation
      ↓
Return verified result
      ↓
    LLM explanation
```

This would be more reliable than asking the LLM to infer numerical results directly from a textual representation of the dataset.

6. Add Statistical Verification

The system could automatically validate model-generated claims using Python.

For example:

LLM:

"Column X appears strongly correlated with Y."

↓

Python:

Calculate Pearson/Spearman correlation

↓

Visualization:

Generate scatter plot

↓

Final:

Verified analytical conclusion

This creates a stronger separation between LLM reasoning and computational truth.

7. Add Conversation Memory

The agent could maintain context across multiple analytical questions.

For example:

User:
Analyze the dataset.

Agent:
Initial findings...

User:
Focus on the customer segment.

Agent:
Segment analysis...

User:
Compare that with last year's data.

Agent:
Comparison...

This would turn the notebook experiment into a more complete conversational data-analysis agent.

8. Add Reproducible Experiment Tracking

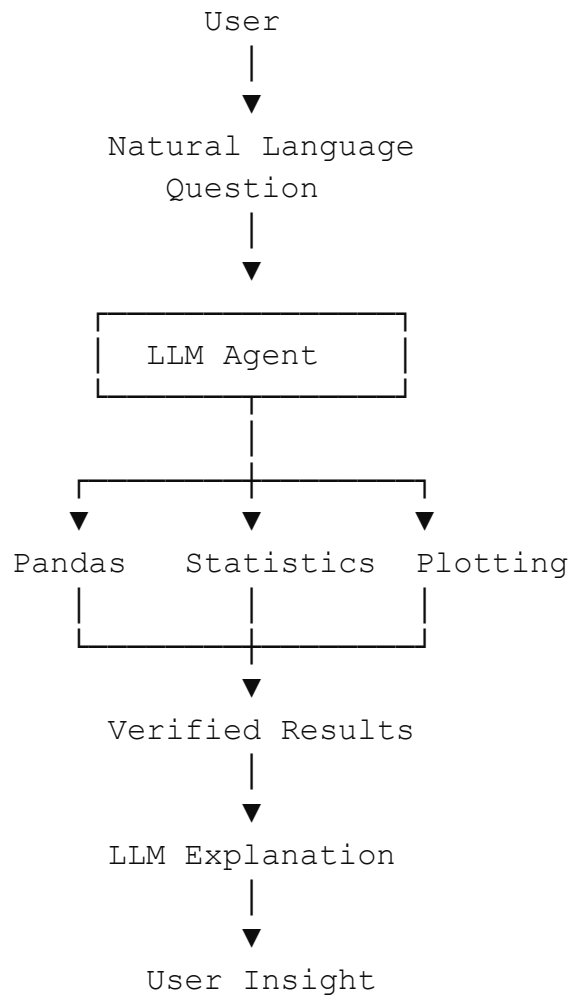
Future versions could record:

- Dataset version
- Model name
- Model parameters
- Prompt
- Analytical question
- Generated response
- Python calculations
- Visualizations
- Execution timestamp

This would make experiments easier to reproduce and audit.

Potential Production Architecture

A future version could evolve into the following system:



This architecture would combine the reasoning capabilities of an LLM with deterministic computational tools.

Conclusion

Self-Driven Agent 1 Working is an experimental exploration of LLM-assisted data science.

The project demonstrates how a large language model can be integrated into a Python notebook to provide a natural-language interface for interacting with tabular data.

Rather than replacing traditional data-science tools, the project explores a hybrid approach where:

LLMs provide reasoning and interaction, while Python-based analytical tools provide computation and verification.

The current notebook serves as a proof of concept for this workflow.